

Enterprise Ontology Lab Workbook

Build, query, reason and explain

Twenty-five notebook walkthroughs with learner exercises, separate solutions and a complete healthcare capstone.

Joe Hoeller
AI Systems & Enterprise Knowledge Engineer

Healthcare learning edition
Real source data · SKOS alignment · BFO · OWL DL
Enterprise products, evidence and adoption

Contents

Chapter titles and page numbers are linked. The outline panel provides another route through the book.

Start here: the executable workbook	3
E00 · Reproduce the source and the environment	4
E01 · Begin with a decision and a product owner	6
E02 · Map records, encounters and people into RDF	8
E03 · Build a controlled vocabulary and a taxonomy	10
E04 · Align taxonomies across code systems	12
E05 · Separate TBox, ABox and RBox	14
E06 · Align to BFO and define upper and lower bounds	16
E07 · Run OWL DL and understand its limits	18
E08 · Validate contracts with SHACL	20
E09 · Bridge a warehouse and an ontology	22
E10 · Query evidence across named graphs	24
E11 · Extend, regress and retract an ontology	26
E12 · Catalog a semantic data product	28
E13 · Deliver the capstone and measure adoption	30
S00 · Inspect the graph before writing a query	32
S01 · Filter literals and bind values	34
S02 · Keep missing values and negation precise	36
S03 · Aggregate at the correct grain	38
S04 · Walk taxonomy paths without claiming DL reasoning	40
S05 · Manage graphs and reversible updates	42
S06 · Query OWL DL results with a stated contract	44
S07 · Inspect blank nodes and OWL lists	46
S08 · Debug portability and application boundaries	48
S09 · Build an explainable query product	50
E14 · Use a small SWRL rule deliberately	52
Capstone review and answer guidance	55

Start here: the executable workbook

This workbook is the practical companion to the primer. It contains 15 enterprise lessons and 10 SPARQL lessons. Each chapter gives the learning outcome, the exact notebook path, an explanation, runnable walkthrough code, a learner task and an explanation prompt.

Run README.md's setup commands once. Open the learner notebooks first. Each exercise begins with `answer = None`; replace it with your code. The check reports an incomplete exercise without pretending it passed. If the answer is wrong, it raises an assertion with a targeted hint. Restart and run all cells after a change to expose hidden notebook state.

Separate solution notebooks contain completed answers and execution outputs. The code cells were executed sequentially in fresh IPython processes because the build environment does not permit Jupyter kernel transports. The standard Jupyter runner is also included for local use. Semantic checks run independently of notebook display.

The clinical subset is published Kaggle data, with source identifiers and values retained. The later code-system tables are reference vocabularies; they do not overwrite historical diagnoses. All deliberate failure experiments alter graph copies. The original source CSV remains unchanged.

The capstone requires working inference, source evidence, a cataloged product and a pilot scorecard. The shared learning outcome is to measure whether people can use the product, alongside checking whether its answers are correct.

Book text and notebooks are CC BY 4.0; original Python utilities are MIT. Data and third-party terms retain their own licenses. Notices and provenance remain in the package.

E00 · Reproduce the source and the environment

Outcome: Verify that the learning graph comes from published clinical records and that every transformation is traceable.

Notebook: notebooks/enterprise/E00_reproduce_the_source_and_the_environment.ipynb

Time: about 30 minutes.

This course uses 60 encounters selected from Kaggle's brandao/diabetes dataset, version 1. The underlying UCI resource describes hospital encounters from 1999–2008. Every selected value is retained in data/encounters.csv. We select 20 rows each for source primary diagnosis tokens 250.02, 428 and 493, ordered by numeric encounter ID. This makes the cohort small enough to inspect and deliberately balanced for learning. It is not a representative hospital sample.

Read the source manifest before querying. Source diagnoses are ICD-9-CM; the later code systems are independent reference layers. Missing and categorical values remain visible. A1Cresult is a category, not a numeric laboratory observation. The source value None records the absence of an HbA1c measurement in the source dictionary; it is not a normal result. A dataset about diabetes cannot support population-wide prevalence estimates.

The core notebooks run offline after installing Python packages and Java 17. RDFLib handles graph storage and SPARQL; pySHACL checks explicit data contracts; OWLAPI checks profiles; HermiT handles OWL DL. Owlready2 supplies the packaged reasoner and JPyPe connects Python to Java. A pinned BFO core is included so imports resolve locally. The requirements-lock file records the environment used to execute the solutions.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Inspect the manifest and verify the selected CSV

```
manifest=json.loads((ROOT/'data/source_manifest.json').read_text())
assert digest(ROOT/'data/encounters.csv') == manifest['subset_sha256']
print({k:manifest[k] for k in
      ↪ ['kaggle_ref', 'kaggle_version', 'source_rows', 'subset_rows', 'source_coding']})
```

Read strings before casting only measured counts

```
import pandas as pd
frame=pd.read_csv(ROOT/'data/encounters.csv',dtype=str,keep_default_na=False)
display(frame.head())
display(frame.groupby('diag_1').size().rename('encounters'))
assert set(frame.diag_1)=='{250.02', '428', '493'}
```

Your turn

Write Python that counts the records with a source readmission category of <30. This is a count in this selected dataset, not a risk model.

```
answer = None # Replace with your solution
```

Hint: Read the category as a string; preserve the less-than sign.

Explain your model

Explain why neither the cohort selection nor a code mapping establishes a causal claim about readmission.

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E01 · Begin with a decision and a product owner

Outcome: Connect a competency question to a decision, evidence requirement and measurable adoption outcome.

Notebook: notebooks/enterprise/E01_begin_with_a_decision_and_a_product_owner.ipynb

Time: about 35 minutes.

The product is an encounter-code review workbench for a terminology steward and an analyst. It retrieves source records, shows their code-family grouping, and proposes cross-version terminology links for review. It does not diagnose a patient or automatically recode a claim. The first business question is: which records fall into the respiratory code family, and which source value explains that answer?

Write competency questions as answerable contracts. Record a consumer, the decision they make, the unit of analysis, the graph scope, the expected answer and a failure condition. Our unit is an encounter record. Returning the right patient with the wrong encounter is a failure. Returning an unsupported replacement ICD-11 code is also a failure.

The learning outcome is explicit: learners will measure whether people can use the product, alongside checking whether its answers are correct. Correctness compares answers with a reviewed reference set. Usability measures whether intended users finish representative tasks, how long it takes, and where they need help. Adoption measures repeated use among eligible users. These denominators differ; a high query count is not proof of usefulness.

Do not fill a value case with unmeasured benefits. Begin with a baseline task observation, estimate a target separately, and record operating and review costs. Measure again with the product. A faster workflow that silently broadens a clinical category has not delivered the intended value.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Write a decision contract

```
contract={
  'consumer':'Terminology steward',
  'decision':'Which source-code mappings need review?',
  'unit':'Encounter record',
  'question':'Which encounters have primary source token 493?',
  'expected_count':20,
  'evidence':'Source encounter ID and original diag_1',
  'owner_role':'Clinical terminology product owner',
  'baseline_minutes':None,
  'pilot_task_completion_rate':None}
display(contract)
```

Separate measured data from future pilot measurements

```
measured_count=sum(r['diag_1']=='493' for r in rows())
assert measured_count==contract['expected_count']
print('Verified source answer:',measured_count)
print('Pilot measures remain uncollected:',contract['pilot_task_completion_rate'] is None)
```

Your turn

Create a dictionary with consumer, decision, baseline, outcome_metric and adoption_metric keys. Leave baseline as None until you observe it.

```
answer = None # Replace with your solution
```

Hint: Specify both an answer-quality metric and a use metric; do not manufacture a baseline.

Explain your model

Teach back the same inference to an engineer and a product owner, using one sentence for each.

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E02 · Map records, encounters and people into RDF

Outcome: Construct triples while keeping real-world identity, record identity and source identifiers distinct.

Notebook: notebooks/enterprise/E02_map_records_encounters_and_people_into_rdf.ipynb

Time: about 40 minutes.

A CSV row is an information record. The hospital encounter it documents is an event unfolding in time. The patient is a person who may participate in several encounters. These deserve separate IRIs. Reusing the patient identifier as the row identifier would collapse repeated admissions. The course uses the source encounter_id and patient_nbr as local identifiers without claiming they are universal identities.

An RDF triple connects subject, predicate and object. Objects may be IRIs or literals. A typed integer supports numeric comparisons, while a string preserves the lexical diagnosis token. Never parse diagnosis codes as floating-point measurements: punctuation and leading zeros can carry code-system meaning. Human labels can change without changing the entity's IRI.

The adapter adds code-family types under an explicit local mapping policy. These are derived categorizations of code concepts, not new diagnoses of patients. The source CSV stays unchanged. A named graph can separate a transformation output from its source manifest, but graph names alone do not grant authorization or establish provenance. The pipeline must record who produced the graph and from which version.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Build three triples by hand

```
r=rows()[0]
g=fresh_graph()
record=record_iri(r)
g.add((record,RDF.type,EX.EncounterRecord))
g.add((record,EX.encounterId,Literal(r['encounter_id'])))
g.add((record,EX.primaryCode,CODE[r['diag_1']]))
print(g.serialize(format='turtle'))
```

Inspect the complete adapter output

```
g=build_asserted()
result=query(g,'SELECT ?id ?code ?days WHERE { ?r ex:encounterId ?id; ex:primaryCode ?code;
  ↳ ex:stayDays ?days } ORDER BY ?id LIMIT 5')
display(list(result))
assert len(members(g,EX.EncounterRecord))==60
```

Your turn

Construct the patient IRI for the first row using its source patient_nbr. Return a URIRef, not a literal.

```
answer = None # Replace with your solution
```

Hint: Use RES and retain the patient identifier as text.

Explain your model

What would break if a warehouse dimension surrogate key were treated as an enduring person identity?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E03 · Build a controlled vocabulary and a taxonomy

Outcome: Use SKOS labels, concept schemes and hierarchy without confusing concepts with OWL classes.

Notebook:

notebooks/enterprise/E03_build_a_controlled_vocabulary_and_a_taxonomy.ipynb

Time: about 40 minutes.

A controlled vocabulary chooses the allowed terms for a task. A taxonomy adds broader and narrower relationships. SKOS makes these structures machine-readable without committing every term to a class of real-world things. In our graph, a code concept is an information resource. It is not a disease episode and it is not the patient described by a record.

Keep one preferred label per language, alternative labels for approved synonyms, a notation for the code string, and a scope note when a label is ambiguous. Use concept schemes to identify distinct curated vocabularies and versions. A broader edge is an immediate hierarchy assertion; use `broaderTransitive` or a property path when you need ancestors. The direct `broader` property itself is not transitive. A polyhierarchy can be useful, but cycles are rejected by this project's editorial rules.

Taxonomy quality includes more than syntax. Check preferred-label uniqueness, label-property overlap, roots, orphan concepts, unexpected cross-scheme edges and cycles. Confirm the meaning of a child is actually within its parent. Similar spelling, a shared department, or co-occurrence is not enough. OWL subclass relationships impose class-inclusion semantics; a SKOS `broader` edge does not automatically become a subclass axiom.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Read the concept scheme

```
v=vocabulary()
display(list(query(v, 'SELECT ?c ?label WHERE { ?c a skos:Concept; skos:prefLabel ?label } ORDER BY
  ↳ ?label')))
assert skos_issues(v)==[]
```

Observe an explicit hierarchy path

```
ancestors=list(query(v, 'SELECT ?parent WHERE { <https://example.org/health/reporting/respiratory>
  ↳ skos:broader+ ?parent }'))
print(ancestors)
assert len(ancestors)==1
```

Your turn

Add a second English preferred label to local:respiratory in a separate graph copy. Return the list of quality issues so you can see the editorial gate fail.

```
answer = None # Replace with your solution
```

Hint: Call `skos_issues` after changing a copy. Do not edit the accepted file.

Explain your model

When should a label be an alternative label instead of a new concept?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E04 · Align taxonomies across code systems

Outcome: Select a mapping relation, preserve its direction and scope, and keep proposals separate from accepted mappings.

Notebook: notebooks/enterprise/E04_align_taxonomies_across_code_systems.ipynb

Time: about 55 minutes.

SKOS alignment relates concepts in different schemes. `exactMatch` is symmetric and transitive, so one mistaken link can spread through a network. `closeMatch` is symmetric but not transitive. `broadMatch` points from a narrower source concept to a broader target; `narrowMatch` reverses that direction. `relatedMatch` expresses an associative mapping without hierarchy. None of these relations means `owl:sameAs`, `owl:equivalentClass` or a patient diagnosis.

Our accepted `broadMatch` links place source code tokens into local reporting families. A local alias with exactly the same scope as source token 493 has an `exactMatch` to that token. The proposed link from source token 493 to ICD-10-CM J45 is represented as a review record. Reifying a statement does not assert its triple. That distinction lets a catalog display the proposal without allowing the query layer to treat it as approved.

The dataset is historical ICD-9-CM. The April 2026 ICD-10-CM vocabulary and January 2025 ICD-11 MMS references are separate releases for comparison. ICD-20 is not the WHO revision used here; ICD-11 is the current revision family. The three ICD-11 references are verified against WHO's release. A crosswalk may be approximate, one-to-many or require clinical context absent from a row. WHO cautions against treating crosswalks as automatic conversion. Do not replace a source diagnosis with a later code on the strength of a similar label.

For each mapping record the two schemes, versions, source and target IRIs, relation direction, rationale, evidence, reviewer role, status and intended use. Evaluate precision on reviewed accepted links, coverage against the source scope, and unresolved cases separately. A confidence score is not an approval.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Separate proposed from accepted statements

```
v=vocabulary()
assert (CODE['493'],SKOS.closeMatch,CM.J45) not in v
proposal=list(v.triples((LOCAL.proposal493,None,None)))
display(proposal)
display(list(query(v,'SELECT ?code ?family WHERE { ?code skos:broadMatch ?family } ORDER BY
  ↳ ?code')))
```

Read actual code reference files

```
import pandas as pd
display(pd.read_csv(ROOT/'data/icd10cm_terms.csv',dtype=str).loc[:,['code','label','release']])
display(pd.read_csv(ROOT/'data/icd11_references.csv',dtype=str).loc[:,['code','label','release']])
assert not any(str(o).startswith(str(CM)) for o in build_asserted().objects(None,EX.primaryCode))
```

Your turn

Write the SPARQL query that returns source concepts and their local broader reporting families. Return the number of result rows.

```
answer = None # Replace with your solution
```

Hint: The direction is source token -> broader reporting family.

Explain your model

What evidence would you need before changing closeMatch to exactMatch? Explain the consequence of transitivity.

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E05 · Separate TBox, ABox and RBox

Outcome: Recognize axioms about classes, individuals and relationships; explain why OWL is not a SQL constraint checker.

Notebook: notebooks/enterprise/E05_separate_tbox_abox_and_rbox.ipynb

Time: about 40 minutes.

TBox axioms describe class relationships and restrictions. ABox axioms describe individuals. RBox is a useful informal name for the property axioms that describe roles, including subproperties, inverses and chains. The labels are a modeling aid; OWL files do not need one physical file per box, and declarations and annotations do not fit neatly into a three-way partition.

The schema says a respiratory-coded record is an EncounterRecord with some primaryCode in RespiratoryCode. The source adapter states that a particular record has source token 493 and that the token belongs to the local respiratory code family. A reasoner combines those premises to classify the record. This is evidence about the coding record, not proof of the patient's current physiological state.

An OWL domain or range says what follows when a property is used. It does not reject a row with an unexpected subject. Several domain statements apply together; they do not mean either domain is allowed. A necessary condition says every member must satisfy a description; it does not classify every thing satisfying that description. Equivalence supplies both directions. Check which direction your business requirement actually needs.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Inspect axioms as RDF structures

```
s=schema()
print('EncounterRecord superclasses:',list(s.objects(EX.EncounterRecord,RDFS.subClassOf)))
print('Primary code domain:',list(s.objects(EX.primaryCode,RDFS.domain)))
print('Property chain heads:',list(s.objects(EX.aboutPatient,OWL.propertyChainAxiom)))
```

Compare asserted and classified memberships

```
asserted=build_asserted()  
report,inferred=reasoning()  
assert report['DL']['in_profile'] and report['consistent']  
print('Asserted review candidates:',len(members(asserted,EX.ReviewCandidate)))  
print('Entailed review candidates:',len(members(inferred,EX.ReviewCandidate)))
```

Your turn

Return a dictionary classifying a subclass axiom, a record type assertion and a property chain into TBox, ABox and RBox.

```
answer = None # Replace with your solution
```

Hint: Focus on the subject of the logical statement, not its Turtle filename.

Explain your model

Give one case where SHACL should reject data that remains consistent under OWL.

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E06 · Align to BFO and define upper and lower bounds

Outcome: Align concrete entities to foundational categories and distinguish architectural layers from logical bounds.

Notebook:

`notebooks/enterprise/E06_align_to_bfo_and_define_upper_and_lower_bounds.ipynb`

Time: about 55 minutes.

BFO distinguishes continuants, which persist through time, from occurrents, which unfold in time. A patient is modeled as an object; a hospital encounter as a process; an encounter record as a generically dependent continuant. The last choice treats the record as information content capable of multiple copies, not as the physical paper or disk carrying it. A code concept is also treated as information content under a local modeling commitment. A SKOS concept is not automatically a BFO universal.

The full pinned `bfo-core.owl` is imported, with its license and commit recorded. It is OWL 2 DL in this tested closure and is not OWL RL. Never infer profile compliance from an ontology's reputation. The learner checks the imported ontology together with the domain schema and facts. CCO or IAO can supply more specific information and organizational categories in a later module; adding their names or IRIs alone would not constitute a reviewed import.

There are two different meanings of upper and lower here. In architecture, upper, mid-level and domain ontologies describe levels of abstraction. Build top-down when shared categories constrain integration; bottom-up when source data and competency questions reveal needed distinctions; meet in the middle by reviewing explicit bridge axioms. In logic, `L subclassOf C subclassOf U` bounds a target class `C`: `L` is sufficient for membership and `U` is necessary. `U` membership alone does not establish `C`. For this lab `RespiratoryCodedRecord` is a lower bound for `ReviewCandidate`; `EncounterRecord` is an upper bound.

Keep approved bridges in a small domain module rather than editing BFO. Review scope, identity, time, disjointness, counterexamples, profile compliance and consistency before accepting a bridge. Never align an encounter record directly to process: the record describes a process but is not that process.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Check actual BFO superclass entailments

```
report,inferred=reasoning()
g=build_asserted()+inferred
first=record_iri(rows()[0])
assert (first,RDF.type,BFO.BFO_0000031) in g
print('BFO import count:',len(report['imports']))
print('Records classified as information content:',len(members(g,BFO.BFO_0000031)))
```

Compute the operational bounds over named source records

```
lower=members(g,EX.RespiratoryCodedRecord)
target=members(g,EX.ReviewCandidate)
upper=members(g,EX.EncounterRecord)
assert lower <= target <= upper
print({'lower':len(lower), 'target':len(target), 'upper':len(upper), 'upper_not_known_target':len(upper-target)})
```

Your turn

Return the BFO IRI that is the chosen superclass of HospitalEncounter. Use the actual ontology term, not its label.

```
answer = None # Replace with your solution
```

Hint: A hospital encounter unfolds through time.

Explain your model

Why is a database row not the same entity as the event that row documents?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E07 · Run OWL DL and understand its limits

Outcome: Classify with HermiT, inspect profile reports, and distinguish graph answers from entailment.

Notebook: notebooks/enterprise/E07_run_owl_dl_and_understand_its_limits.ipynb

Time: about 55 minutes.

OWL 2 DL reasoning is model-theoretic: an entailment holds in every interpretation satisfying the ontology. It is not a traversal of a few subclass edges. The imported BFO core and existential restrictions in this course require a DL-capable path. The RL exercise remains useful on a separately restricted module; owlrl is not a replacement for HermiT on this input.

Run profile validation before classification and consistency before returning answers. An unsatisfiable class cannot have members in any model but an ontology containing an empty unsatisfiable class can remain consistent. Asserting an instance of that class makes the ontology inconsistent. Different names do not automatically denote different individuals. A maximum-cardinality restriction can force equality, whereas an explicit inequality can reveal a contradiction.

Our exporter asks HermiT for named class memberships of the named individuals. It does not implement a complete SPARQL OWL Direct Semantics endpoint. Arbitrary joins over unnamed existential witnesses, negative facts and inferred object-property values are outside the exported query contract. SPARQL over the selected export is useful precisely because its coverage is stated. Count returned named records, not all possible objects in a model.

For an existential restriction, a reasoner may conclude that a suitable object exists without assigning it a public IRI. A missing documents edge can therefore be consistent in OWL while failing the SHACL requirement that the edge be supplied. Do not manufacture a named witness merely to make an export look complete.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Read the profile and class results

```
report,inferred=reasoning()
display({k:report[k] for k in ['engine','scope','consistent','unsatisfiable','export_coverage']})
assert report['DL']['in_profile'] and not report['RL']['in_profile']
assert len(members(inferred,EX.CardiopulmonaryRecord))==40
```

Remove a required explicit edge in a local experiment

```
changed=build_asserted()
record=record_iri(rows()[0])
changed.remove((record,EX.documents,None))
dl_report,_=reasoning(source=changed)
print('OWL consistent:',dl_report['consistent'])
print('SHACL conforms:',validate(changed)[0])
assert dl_report['consistent'] and not validate(changed)[0]
```

Compare a restricted RL module

```
from owlrl import DeductiveClosure, OWLRL_Semantics
rl=Graph()
rl.add((EX.ReviewCandidate,RDF.type,OWL.Class))
rl.add((EX.EncounterRecord,RDF.type,OWL.Class))
rl.add((EX.ReviewCandidate,RDFS.subClassOf,EX.EncounterRecord))
record=record_iri(next(r for r in rows() if r['diag_1']=='493'))
rl.add((record,RDF.type,EX.ReviewCandidate))
assert (record,RDF.type,EX.EncounterRecord) not in rl
DeductiveClosure(OWLRL_Semantics).expand(rl)
assert (record,RDF.type,EX.EncounterRecord) in rl
print('RL derives the superclass in this restricted module; it does not replace the full DL run.')
```

Your turn

Return the number of named cardiopulmonary-coded records entailed by the supplied ontology.

```
answer = None # Replace with your solution
```

Hint: Use the named class memberships returned by the reasoner.

Explain your model

Why is absence from a SPARQL result not an OWL proof of negation?

E08 · Validate contracts with SHACL

Outcome: Separate intake validation, logical consistency and serving-graph checks.

Notebook: notebooks/enterprise/E08_validate_contracts_with_shacl.ipynb

Time: about 40 minutes.

An enterprise data contract should make omissions and datatype errors actionable. SHACL validates the graph provided to it against declared shapes. This lab validates the explicit intake graph with inference set to none. Requiring one primary code and one source encounter identifier is a delivery rule. It says what the producer must supply even when open-world logic could allow more information elsewhere.

The stayDays range is grounded in the source cohort's inclusion criteria, not a universal clinical law. A historical system with a different cohort could need a different shape. Test shapes with deliberately altered copies of real records: remove an identifier, add a second code, or change a typed count to text. Keep the original CSV and accepted graph unchanged.

A serving shape may differ from an intake shape. If a consumer expects an inferred ReviewCandidate type, validate the explicit-plus-approved-derived graph. Name the target class and inference policy so the same data does not pass in one deployment and fail in another for an invisible reason. Validation reports are RDF and can be cataloged with the run's provenance and model version.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Read the baseline validation report

```
conforms,report_graph,report_text=validate()
assert conforms
print(report_text)
print('Report triples:',len(report_graph))
```

Inject one missing identifier and locate its violation

```
bad=build_asserted()  
target=record_iri(rows()[0])  
bad.remove((target,EX.encounterId,None))  
conforms,issues,text=validate(bad)  
assert not conforms  
print(text)
```

Your turn

On a new graph copy, set stayDays to the integer 99 for the first record. Return the conformance boolean.

```
answer = None # Replace with your solution
```

Hint: Use Graph.set to replace the one value; 99 exceeds this cohort contract.

Explain your model

Which version should change if the dataset expands to encounters longer than 14 days?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E09 · Bridge a warehouse and an ontology

Outcome: Reconcile SQL and SPARQL at the same grain, with explicit treatment of identifiers and missing values.

Notebook: notebooks/enterprise/E09_bridge_a_warehouse_and_an_ontology.ipynb

Time: about 50 minutes.

The relational table is useful evidence, not an obstacle to semantic modeling. Start with the table's grain and candidate keys. Here the `encounter_id` identifies one row. The person and encounter are modeled separately, and counts are cast only after reading the source codes as strings. A type-2 slowly changing dimension row would need its own version identity as well as a link to the enduring entity.

SQL constraints and OWL axioms have different roles. A database unique constraint rejects duplicate keys; OWL functionality can imply that two fillers denote the same individual. SQL NULL and absent RDF values are not interchangeable truth values. The adapter must choose what a missing source token means. Aggregation counts solution rows in a selected graph; logical entailment decides whether a proposition follows.

Use reconciliation to discover hidden modeling changes. Compare the same population, filters, time window, units and code versions. A SQL count on source rows and a SPARQL count after a one-to-many label join can disagree even when neither parser reports an error. R2RML gives a standard mapping vocabulary; the supplied mapping file illustrates the subject template, logical table and predicate-object maps. The Python adapter is the executed implementation, not an R2RML processor.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Ask equivalent SQL and SPARQL questions

```
db=sql_connection()
sql=dict(db.execute('SELECT diag_1, COUNT(*) FROM encounters GROUP BY diag_1'))
g=build_asserted()
sparql={str(c).rsplit('/',1)[-1]:int(n) for c,n in query(g,'SELECT ?c (COUNT(?r) AS ?n) WHERE { ?r
  ↳ a ex:EncounterRecord; ex:primaryCode ?c } GROUP BY ?c')}
assert sql==sparql=={'250.02':20,'428':20,'493':20}
display(sql)
```

Inspect the declared R2RML bridge

```
print((ROOT/'ontology/encounter-mapping.ttl').read_text())
print('Physical key -> record IRI:',rows()[0]['encounter_id'],record_iri(rows()[0]))
```

Your turn

Write SQL to count source rows with `time_in_hospital` at least 7; compare with a SPARQL FILTER on `ex:stayDays`. Return a pair of equal counts.

```
answer = None # Replace with your solution
```

Hint: Both counts must use encounters, not patients.

Explain your model

How would a join to several alternate labels change a naive `COUNT(?r)`?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E10 · Query evidence across named graphs

Outcome: Publish a finite entailment view with source-level evidence and explicit graph scope.

Notebook: notebooks/enterprise/E10_query_evidence_across_named_graphs.ipynb

Time: about 40 minutes.

Use named graphs to make query scope visible. This dataset has asserted transformation output, vocabulary, selected derived memberships and metadata. The default graph is empty by design. A query without GRAPH is therefore not a shortcut to the union. Store products differ on default-union behavior, so this choice belongs in the data product's contract.

An answer should join a derived review classification back to the original source identifier and code. The explanation is a small proof sketch: the record has code 493; the adapter places that code in RespiratoryCode; the equivalent-class definition identifies a RespiratoryCodedRecord; the approved subclass axiom makes it a ReviewCandidate. This sketch is not an automatically minimal proof from the reasoner. Preserve the supporting rule and transform versions so another analyst can reproduce it.

Named graphs are useful packaging boundaries, not a security system. Authorization must apply to evidence as well as answers. A graph query that reveals a hidden source identifier through an inferred label still discloses information. This teaching product uses public data; an enterprise deployment must implement the relevant access policy at the service and export layers.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Build a dataset with separate inference output

```
report,inferred=reasoning()
ds=dataset(inferred)
q='''SELECT ?id ?code WHERE {
  GRAPH <urn:graph:derived:v1> { ?r a ex:ReviewCandidate }
  GRAPH <urn:graph:asserted:v1> { ?r ex:encounterId ?id; ex:primaryCode ?code }
} ORDER BY ?id'''
answers=list(query(ds,q))
display(answers[:5]);assert len(answers)==20
```

Check the empty default graph explicitly

```
assert len(list(query(ds,'SELECT ?s WHERE { ?s a ex:EncounterRecord }')))==0
print('20 review records have source evidence; default graph contains no intake records.')
```

Your turn

Return all source encounter identifiers that explain the 20 review candidates, as a set of strings.

```
answer = None # Replace with your solution
```

Hint: Use the graph-scoped evidence query; do not join by labels.

Explain your model

What extra metadata would you need to reproduce the answer six months after a model update?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E11 · Extend, regress and retract an ontology

Outcome: Make a bounded extension and rebuild dependent conclusions after a change.

Notebook: notebooks/enterprise/E11_extend_regress_and_retract_an_ontology.ipynb

Time: about 50 minutes.

A safe extension has a competency question, reviewed definitions, a profile check, a consistency check and answer regression. Extending an ontology is more than minting a class. A new subclass axiom can change every consumer that queries its superclass. An import update can add consequences outside the edited file. Version the ontology and its approved closure as one deployable dependency.

The cardiac extension broadens the review workflow from respiratory-coded records to cardiac-coded records as well. It adds one subclass axiom in a local extension file. The operational rule is a review policy, not a new medical inference. The expected review set changes from 20 to 40 and the original 20 remain. That is an intentional semantic change which requires an updated product contract.

OWL entailment is monotonic for a fixed language and growing premise set: adding axioms does not invalidate earlier logical consequences. Removing or correcting premises is different. A stored materialization can keep obsolete consequences unless it is retracted or rebuilt. This small course rebuilds from accepted source facts and the chosen model version; it does not pretend to implement incremental truth maintenance.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Measure the extension impact

```
base_report,base=reasoning()
new_report,expanded=reasoning(extension=True)
before=members(base,EX.ReviewCandidate);after=members(expanded,EX.ReviewCandidate)
assert before<=after and len(before)==20 and len(after)==40
print({'before':len(before),'after':len(after),'newly_included':len(after-before)})
```

Delete a supporting fact and rebuild

```
edited=build_asserted()
removed=next(r for r in rows() if r['diag_1']=='493')
edited.remove((record_iri(removed),EX.primaryCode,None))
rep,rebuilt=reasoning(source=edited)
assert len(members(rebuilt,EX.ReviewCandidate))==19
assert str(record_iri(removed)) not in members(rebuilt,EX.ReviewCandidate)
print('Rebuilt named candidates:',len(members(rebuilt,EX.ReviewCandidate)))
```

Your turn

Return the set difference identifying the 20 newly included record IRIs after the cardiac extension.

```
answer = None # Replace with your solution
```

Hint: Compare identities rather than only the two totals.

Explain your model

Why should a rollout gate compare actual answer sets and not only their cardinality?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E12 · Catalog a semantic data product

Outcome: Package data, model versions, quality reports and service expectations for discovery and reuse.

Notebook: notebooks/enterprise/E12_catalog_a_semantic_data_product.ipynb

Time: about 40 minutes.

A reusable semantic product needs a named consumer and owner, a discoverable entry, an access policy, freshness and quality expectations, a versioned schema, support procedures and a cost model. DCAT describes datasets, distributions and data services. PROV-O expresses derivation and processing lineage. Neither vocabulary supplies an authorization mechanism or guarantees the metadata is true.

The course creates a real catalog entry for the included CSV distribution. Its download locator is a local artifact URN, because there is no deployed public service. A production catalog would replace that locator with an actual accessible distribution URL and add the relevant DataService entry. Advertising a nonexistent endpoint as running would break the product contract.

Publish source and ontology checksums, transformation version, reasoner configuration, accepted mapping set, validation result and run time with each release. Distinguish source vintage from refresh cadence: regularly rebuilding a dataset from 1999–2008 does not make the clinical observations current. Product adoption also depends on documentation, training and meaningful feedback channels; metadata completeness alone does not establish use.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Create and query catalog metadata

```
cat=catalog()
print(cat.serialize(format='turtle'))
product=RES['product/encounter-code-review']
assert (product,RDF.type,DCAT.Dataset) in cat
assert len(list(cat.objects(product,DCAT.distribution)))==1
```

Inspect a product contract template

```
product_contract=json.loads((ROOT/'docs/product_contract.json').read_text())
display(product_contract)
assert product_contract['deployment_state']=='local learning package'
assert product_contract['live_endpoint'] is None
```

Your turn

Return the SHA-256 of the actual included encounter CSV, as stored in the catalog distribution.

```
answer = None # Replace with your solution
```

Hint: Query the distribution metadata; use the actual file checksum.

Explain your model

What should the product team do when a consumer reports that a correct answer is hard to interpret?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E13 · Deliver the capstone and measure adoption

Outcome: Deliver a working inference, evidence, a cataloged product and a pilot measurement plan.

Notebook: notebooks/enterprise/E13_deliver_the_capstone_and_measure_adoption.ipynb

Time: about 70 minutes.

The capstone brings the system together: parse source data; check the intake contract; build the logical import closure; validate the OWL profile; check consistency; classify named individuals; publish the chosen finite entailment view; reconcile expected answers; produce an explanation; and describe the product in a catalog. A failure at a gate should stop publication, not become an empty success result.

The technical scorecard checks source integrity, answer identity, required evidence, profile compliance, consistency and reproducible rebuilding. The adoption scorecard observes representative users completing realistic tasks. Ask them to locate a record, explain its review status, identify the original diagnosis token, and recognize an unapproved mapping. Record assistance, elapsed time and errors. Use feedback to change labels, evidence displays or training, then repeat the same task protocol.

Calculate task completion as successful tasks divided by attempted tasks. Track answer correctness separately, and report review time and operating cost with their denominators. Weekly active or returning users must be divided by the eligible user group, not every account in the organization. Do not publish adoption percentages until a pilot actually runs. A blank measurement field is more honest and more useful than an unmeasured result.

Your release review has four deliverables: working inference; explanation with source evidence; catalog entry and product contract; and a pilot scorecard covering correctness, task completion, review time, adoption and cost. A reviewer should be able to rerun the notebooks and explain one answer to a business stakeholder without reading the implementation.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Run the release gates

```
assert validate()[0]
report, derived=reasoning()
assert report['DL']['in_profile'] and report['consistent'] and not report['unsatisfiable']
expected={str(record_iri(r)) for r in rows() if r['diag_1']=='493'}
actual=members(derived, EX.ReviewCandidate)
assert actual==expected
score={'source_integrity':digest(R00T/'data/encounters.csv')==json.loads((R00T/'data/source_manifests'
↳ t.json').read_text())['subset_sha256'],
        'expected_review_records':len(expected), 'returned_review_records':len(actual),
        'answer_set_correct':actual==expected, 'pilot_completed':False}
display(score)
```

Open the human-use scorecard

```
pilot=json.loads((R00T/'docs/pilot_scorecard.json').read_text())
display(pilot)
assert pilot['measurements']['task_completion_rate'] is None
```

Your turn

Return a release decision dictionary with `publish_learning_artifact=True` and `publish_clinical_service=False`, plus a reason explaining the evidence boundary.

```
answer = None # Replace with your solution
```

Hint: A tested teaching product does not establish a tested clinical deployment.

Explain your model

Present the four deliverables to a peer. Ask them to reproduce a source answer and identify one unsupported mapping.

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S00 · Inspect the graph before writing a query

Outcome: Use small graph views, term inspection and basic SELECT patterns to understand the data.

Notebook: notebooks/sparql/S00_inspect_the_graph_before_writing_a_query.ipynb

Time: about 30 minutes.

Start by identifying the dataset and its grain. Do not write a patient count when the model stores encounter records. Inspect predicates and types on one record, then expand to all records. SELECT returns solution mappings, ASK a boolean, CONSTRUCT a graph and DESCRIBE an implementation-selected description. Use explicit CONSTRUCT templates when downstream consumers need predictable triples.

The original SPARQL course's 19 topics are retained through the topic crosswalk. The pwin course contributes a useful teaching rhythm: inspect a small graph, predict a result, run a query, explain the difference, and check expected values. We apply those techniques to the published healthcare subset. External browser tools and servers are optional; the reproducible core uses RDFLib locally.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Inspect one subject

```
g=build_asserted();subject=record_iri(rows()[0])
display(list(g.predicate_objects(subject)))
display(list(query(g,'SELECT DISTINCT ?p WHERE { ?s ?p ?o } ORDER BY ?p')))
```


Run SELECT, ASK and CONSTRUCT

```
selected=list(query(g,'SELECT ?id WHERE { ?r ex:encounterId ?id } ORDER BY ?id LIMIT 5'))
assert len(selected)==5
assert bool(query(g,'ASK { ?r a ex:EncounterRecord }'))
view=query(g,'CONSTRUCT { ?r ex:encounterId ?id } WHERE { ?r ex:encounterId ?id }').graph
assert len(view)==60
display(selected)
```

Your turn

Return a SPARQL count of EncounterRecord subjects.

```
answer = None # Replace with your solution
```

Hint: COUNT counts matched rows. Here each record has one explicit type match.

Explain your model

Why does LIMIT without ORDER BY fail to define a repeatable first page?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S01 · Filter literals and bind values

Outcome: Use standard SPARQL expressions with explicit datatypes and safe parameter binding.

Notebook: notebooks/sparql/S01_filter_literals_and_bind_values.ipynb

Time: about 40 minutes.

Code strings and measurements have different semantics. The source token 250.02 is not a decimal value. Stay duration is an integer. Compare literals with compatible datatypes and preserve language tags on vocabulary labels. Use STR when you need a lexical form and DATATYPE or LANG when inspecting the term itself.

Standard SPARQL 1.1 uses BIND and COALESCE. LET and IFNULL are not portable SPARQL 1.1 syntax. COALESCE chooses the first expression that evaluates without an error; describing it simply as SQL's non-null selection hides the difference between unbound variables and literal values. Use RDFLib initBindings for trusted local variable binding; do not interpolate raw user text into query syntax.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Filter a typed measurement

```
g=build_asserted()
display(list(query(g,'SELECT ?id ?days WHERE { ?r ex:encounterId ?id; ex:stayDays ?days
  ↳ FILTER(?days >= 7) } ORDER BY DESC(?days) ?id')))
```

Bind a code and compute a label

```
q='''SELECT ?id ?band WHERE {
  ?r ex:encounterId ?id; ex:primaryCode ?wanted; ex:stayDays ?d .
  BIND(IF(?d >= 7, "longer stay", "shorter stay") AS ?band)
} ORDER BY ?id'''
result=list(query(g,q,initBindings={'wanted':CODE['493']}))
assert len(result)==20
display(result[:5])
```

Your turn

Write a query that counts records whose a1cCategory literal is >8. Return the count.

```
answer = None # Replace with your solution
```

Hint: The whole source category is the string >8, not a numeric laboratory value.

Explain your model

When would lexical sorting of encounter IDs differ from numeric sorting?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S02 · Keep missing values and negation precise

Outcome: Predict OPTIONAL and FILTER behavior and distinguish MINUS from NOT EXISTS.

Notebook: notebooks/sparql/S02_keep_missing_values_and_negation_precise.ipynb

Time: about 50 minutes.

An OPTIONAL pattern preserves a solution from the left even when the optional part has no match. A FILTER inside OPTIONAL limits the optional match; a FILTER outside it can discard the whole row. If a comparison encounters an unbound variable it does not evaluate to a successful filter. This small placement choice often changes an operational cohort.

NOT EXISTS tests the absence of a matching graph pattern using the current solution's bindings. MINUS removes compatible solutions and does not remove anything when the two sides have no shared variables. Neither operator proves that an OWL class is false. They operate on the chosen query dataset and entailment policy.

The pwin optionality and negation exercises inspired these paired experiments. We use the real absence of A1C results in the selected source records to make the answer differences visible. Keep the original source token available for audit; lack of a result is not evidence of a normal value.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Move one FILTER and compare row counts

```
g=build_asserted()
inside=list(query(g,'SELECT ?r ?a WHERE { ?r a ex:EncounterRecord OPTIONAL { ?r ex:alcCategory ?a
  ↳ FILTER(?a = ">8") } }'))
outside=list(query(g,'SELECT ?r ?a WHERE { ?r a ex:EncounterRecord OPTIONAL { ?r ex:alcCategory ?a
  ↳ } FILTER(?a = ">8") }'))
assert (len(inside),len(outside))==(60,13)
print({'filter_inside':len(inside),'filter_outside':len(outside)})
```

Compare negation with and without shared variables

```
missing=list(query(g,'SELECT ?r WHERE { ?r a ex:EncounterRecord FILTER NOT EXISTS { ?r  
  ↳ ex:alcCategory ?a } }'))  
minus=list(query(g,'SELECT ?r WHERE { ?r a ex:EncounterRecord MINUS { ?other ex:alcCategory ?a }  
  ↳ }'))  
assert (len(missing),len(minus))== (42,60)  
print({'missing_result':len(missing),'minus_without_shared_variables':len(minus)})
```

Your turn

Write MINUS with the shared variable ?r so it finds records with no alcCategory. Return the count.

```
answer = None # Replace with your solution
```

Hint: Use the same record variable on both sides.

Explain your model

What would happen if you changed the default graph to a union containing newer source records?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S03 · Aggregate at the correct grain

Outcome: Prevent duplicate joins and distinguish COUNT(*) from COUNT of a bound value.

Notebook: notebooks/sparql/S03_aggregate_at_the_correct_grain.ipynb

Time: about 40 minutes.

SPARQL results are multisets. Joining a record to several related terms can produce several rows for one record. COUNT(?r) then counts appearances. COUNT(DISTINCT ?r) counts unique RDF terms, which is often the intended record count. It does not automatically merge two names that a reasoner considers equal unless the query implementation exposes that equivalence.

COUNT(?value) ignores unbound values. COUNT(*) counts the surviving solution row even when an OPTIONAL value is absent. Use this distinction to report both encounters and encounters with a recorded A1C category. For more complex products, preaggregate each independent one-to-many branch in a subquery and join the grouped results at their common grain.

GROUP_CONCAT ordering is not portable unless a specific implementation contract guarantees it. SAMPLE chooses an arbitrary member. Do not use either as a deterministic evidence selection rule. A result that happens to stay stable during a small local run is not a specification guarantee.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Count all rows and available values

```
g=build_asserted()
result=list(query(g,'SELECT (COUNT(*) AS ?rows) (COUNT(?a) AS ?available) WHERE { ?r a
  ↳ ex:EncounterRecord OPTIONAL { ?r ex:alcCategory ?a } }'))
assert tuple(int(v) for v in result[0])==(60,18)
display(result)
```

See a cross-product inflate the count

```
inflated=list(query(g,'SELECT (COUNT(?r) AS ?n) (COUNT(DISTINCT ?r) AS ?unique) WHERE { ?r a  
  ↳ ex:EncounterRecord . VALUES ?view { "source" "review" } }'))  
assert tuple(int(v) for v in inflated[0])==(120,60)  
display(inflated)
```

Your turn

Return the three code-group counts using GROUP BY and COUNT(DISTINCT ?r), as a dictionary keyed by the last segment of the code IRI.

```
answer = None # Replace with your solution
```

Hint: Group by the code IRI; do not join labels before counting.

Explain your model

How would you show zero counts for allowed categories with no encounters?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S04 · Walk taxonomy paths without claiming DL reasoning

Outcome: Use inverse, alternative and transitive paths while keeping reachability distinct from logical entailment.

Notebook:

`notebooks/sparql/S04_walk_taxonomy_paths_without_claiming_dl_reasoning.ipynb`

Time: about 50 minutes.

A property path navigates edges in a selected graph. A plus means one or more steps; a star includes a zero-length path. A slash composes paths, a vertical bar selects alternatives, and a caret reverses direction. Use hierarchy paths to retrieve ancestors without asserting new direct broader edges.

Paths can answer some questions that also have a reasoning formulation, such as transitive ancestry. They do not implement cardinality reasoning, disjointness checking or arbitrary OWL class descriptions. A path over `rdf:type/rdfs:subClassOf*` can miss a classification supported by restrictions and property values. The query's apparent sophistication does not change the entailment regime.

SKOS mapping paths need additional care. `exactMatch` is transitive by its semantics but `closeMatch` is not. Writing `closeMatch+` asks a reachability question and does not make its endpoints a justified `closeMatch`. The `pwin` paths and inference lessons motivate this distinction; the revised course always declares whether it is navigating stored edges or asking a reasoner.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Compare zero-or-more and one-or-more

```
v=vocabulary()
start='<https://example.org/health/reporting/respiratory>'
star=list(query(v,f'SELECT ?x WHERE {{ {start} skos:broader* ?x }}'))
plus=list(query(v,f'SELECT ?x WHERE {{ {start} skos:broader+ ?x }}'))
assert (len(star),len(plus))==2,1)
display(star)
```

Compare path matching with OWL classification

```
g=build_asserted()+schema()
path_answer=list(query(g,'SELECT ?r WHERE { ?r rdf:type/rdfs:subClassOf* ex:ReviewCandidate }'))
report,derived=reasoning()
assert len(path_answer)==0 and len(members(derived,EX.ReviewCandidate))==20
print('Stored subclass path:',len(path_answer),'DL named
↳ classification:',len(members(derived,EX.ReviewCandidate)))
```

Your turn

Use an inverse broader path to find the narrower concept under ICD-10-CM J45. Return its IRI.

```
answer = None # Replace with your solution
```

Hint: The stored broader edge goes from the child to J45.

Explain your model

Why would `closeMatch+` be inappropriate as a rule for automatic code replacement?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S05 · Manage graphs and reversible updates

Outcome: Scope reads and updates explicitly, distinguish an empty graph from a deleted graph, and rebuild derived views.

Notebook: notebooks/sparql/S05_manage_graphs_and_reversible_updates.ipynb

Time: about 40 minutes.

SPARQL query and update use different protocol operations. INSERT DATA adds explicit triples; DELETE DATA removes named ground triples; DELETE/INSERT WHERE transforms matched solutions. CLEAR removes graph contents, while DROP removes the graph from the graph store where the store represents that distinction. Avoid a helper called `clear_dataset` if it only deletes the default graph.

Work in a temporary graph for an exercise. Never run a broad deletion against a production endpoint just to start a tutorial. Updates can invalidate stored inference outputs and quality reports. Version the source snapshot and rebuild or retract dependent conclusions before publishing a new serving graph.

The local notebook demonstrates graph-scoped changes on a fresh in-memory dataset. The `pwin` graph-management exercises inform the sequence. Exported TriG preserves graph boundaries; Turtle does not represent a whole dataset's named graphs.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Inspect named graph scope

```
ds=dataset()
display(list(query(ds,'SELECT ?g (COUNT(*) AS ?n) WHERE { GRAPH ?g { ?s ?p ?o } } GROUP BY ?g ORDER
↳ BY ?g')))
assert len(list(query(ds,'SELECT ?r WHERE { ?r a ex:EncounterRecord }')))==0
```

Insert then clear an explicit work graph

```
work='urn:graph:work'
ds.update(PREFIX+'INSERT DATA { GRAPH <urn:graph:work> { <urn:review:1> ex:status "pending" } }')
assert len(ds.graph(URIRef(work)))==1
ds.update('CLEAR GRAPH <urn:graph:work>')
assert len(ds.graph(URIRef(work)))==0
assert len(members(ds.graph(ASSERTED),EX.EncounterRecord))==60
```

Your turn

Count encounter records in the asserted named graph using GRAPH, and return 60.

```
answer = None # Replace with your solution
```

Hint: Name the graph in the query; the default graph is empty.

Explain your model

What is the impact of changing a source graph while retaining yesterday's derived graph?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S06 · Query OWL DL results with a stated contract

Outcome: Distinguish ordinary SPARQL, exported entailments and a complete DL query regime.

Notebook: notebooks/sparql/S06_query_owl_dl_results_with_a_stated_contract.ipynb

Time: about 55 minutes.

The SPARQL 1.1 entailment-regimes recommendation defines conditions for OWL Direct Semantics evaluation. A normal RDFLib graph or an unconfigured Fuseki dataset does not acquire those semantics merely because it contains OWL triples. SPARQL-DL is also a separate query-language and engine approach, not a synonym for every SPARQL query over an ontology.

Our tested route is explicit: check the complete local imports closure, run HermiT, export entailed named class memberships for the named individuals, then query that finite view with ordinary SPARQL. This is sufficient for the stated review-class questions.

Object-property entailments, negative assertions, arbitrary class-expression queries and unnamed witnesses are not exported by this adapter. A consumer requesting them needs a direct reasoner API or an endpoint with a verified entailment contract.

A useful query contract lists allowed graph patterns, ontology and data versions, import closure, consistency policy, reasoning engine and answer scope. Record expected positive and negative regression cases. If the ontology is inconsistent, stop serving its entailed answers. More triples do not make a contradictory model trustworthy.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Compare an asserted query with a query over named entailed types

```
asserted=build_asserted()
report,derived=reasoning()
before=list(query(asserted,'SELECT ?r WHERE { ?r a ex:ReviewCandidate }'))
after=list(query(asserted+derived,'SELECT ?r WHERE { ?r a ex:ReviewCandidate }'))
assert (len(before),len(after))==0,20)
print(report['export_coverage'])
```

Show that export coverage excludes property-chain consequences

```
combined=asserted+derived
assert len(list(combined.triples((None,EX.aboutPatient,None))))==0
print('No aboutPatient triples are exported; that is a coverage limit, not a non-entailment
↳ claim.')
```

Your turn

Return a query contract with coverage set to `named_class_membership` and `consistency_required` set to `True`.

```
answer = None # Replace with your solution
```

Hint: Be exact about what is materialized.

Explain your model

What evidence would you require before claiming a service implements arbitrary OWL DL conjunctive queries?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S07 · Inspect blank nodes and OWL lists

Outcome: Read the RDF serialization of restrictions and lists without confusing that structure with its logical meaning.

Notebook: notebooks/sparql/S07_inspect_blank_nodes_and_owl_lists.ipynb

Time: about 40 minutes.

OWL class expressions are often serialized using blank nodes and RDF collections. A blank-node identifier is local to a parse or graph and is not a durable business key. Use structural patterns, graph isomorphism or stable source identifiers for comparison rather than persisting an arbitrary blank-node label.

The list path `rdf:rest*/rdf:first` retrieves members of a serialized collection. For `owl:unionOf` those members describe alternatives; the RDF list structure alone does not classify individuals. An anonymous class is still queryable as an RDF node using variables and structural patterns. Giving the expression a named equivalent class can make consumer queries easier, but lack of a name does not make it impossible to query.

This lesson corrects an overstatement in the `pwin` inference commentary: anonymous union classes can be inspected and matched. It also retains the useful lesson that the domain of a property is an inference rule, not an input validation constraint. Keep list inspection, RDFS entailment and OWL DL interpretation conceptually separate.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Traverse the union expression as RDF

```
g=schema()
query_text='''PREFIX owl: <http://www.w3.org/2002/07/owl#>
SELECT ?member WHERE {
  ex:CardiopulmonaryRecord owl:equivalentClass ?expr .
  ?expr owl:unionOf/rdf:rest*/rdf:first ?member .
} ORDER BY ?member'''
union_members={str(row[0]) for row in query(g,query_text)}
assert union_members=={str(EX.CardiacCodedRecord),str(EX.RespiratoryCodedRecord)}
display(union_members)
```

Round-trip without relying on blank-node labels

```
from rdflib.compare import isomorphic
roundtrip=Graph().parse(data=g.serialize(format='turtle'),format='turtle')
assert isomorphic(g,roundtrip)
print('Graph structure is preserved; blank-node spelling need not be.')
```

Your turn

Return the two property IRIs that occur in the aboutPatient property chain.

```
answer = None # Replace with your solution
```

Hint: This query retrieves members; use explicit rdf:first/rest positions if order matters.

Explain your model

Why is a property chain ordered even though the set returned by a path query is not an ordered list?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S08 · Debug portability and application boundaries

Outcome: Separate syntax, data, semantics and implementation issues when a query behaves unexpectedly.

Notebook: notebooks/sparql/S08_debug_portability_and_application_boundaries.ipynb

Time: about 45 minutes.

Debug a query in layers. First parse it. Next confirm the dataset and graph. Inspect IRIs, language tags and datatypes. Count intermediate solution rows before adding OPTIONAL, aggregation or paths. Then check the configured entailment regime and engine-specific extensions. Compare actual bindings, not only row counts.

The pwin course's engine comparisons are useful evidence of how portability problems arise; they are not a guarantee that this package has tested the same engines. This package's execution report names its own tested environment. RDF 1.2, SPARQL 1.2 and GeoSPARQL are extension study topics here, outside the SPARQL 1.1 core execution contract. Do not feed unsupported triple-term syntax or extension functions to a DL parser and claim success.

Federation needs operational controls: an allowlist of destinations, timeouts, a clear policy on sending bound values, stable remote term identities and visible partial failures. SERVICE SILENT can preserve rows after a remote failure but can conceal incomplete enrichment. The course keeps network federation optional so credentials and endpoint availability do not block learning. The HTTP helper separates query, update and graph responses and raises failures instead of presenting them as empty answers.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Parse standard syntax and detect a nonstandard form

```
from rdflib.plugins.sparql.parser import parseQuery
good=PREFIX+ 'SELECT ?r ?band WHERE { ?r ex:stayDays ?d BIND(IF(?d >= 7,"longer","shorter") AS
    ↳ ?band) }'
parseQuery(good)
rejected=False
try:parseQuery('SELECT ?x WHERE { LET (?x := 1) }')
except Exception:rejected=True
assert rejected
print('Portable query parsed; nonstandard LET rejected.')
```

Compare equivalent answer sets

```
g=build_asserted()
q1='SELECT ?r WHERE { ?r ex:primaryCode <https://example.org/health/icd9-source/493> }'
q2='SELECT ?r WHERE { ?r ex:primaryCode ?c VALUES ?c { <https://example.org/health/icd9-source/493>
    ↳ } }'
a={str(r[0]) for r in query(g,q1)};b={str(r[0]) for r in query(g,q2)}
assert a==b and len(a)==20
print('Equivalent named answers:',len(a))
```

Your turn

Return a debugging sequence list containing syntax, dataset, terms, joins and entailment in that order.

```
answer = None # Replace with your solution
```

Hint: Start with checks that can be resolved locally.

Explain your model

Why is changing an HTTP failure into an empty result dangerous for a data product?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

S09 · Build an explainable query product

Outcome: Assemble a reusable query, its expected identities, evidence and user-task acceptance test.

Notebook: notebooks/sparql/S09_build_an_explainable_query_product.ipynb

Time: about 55 minutes.

A query product has consumers and a service contract. Define the answer fields, their interpretation, graph scope, inference coverage, ordering, expected runtime, failure policy and version dependencies. A source code, a mapped concept and an inferred type should appear as different fields when users need to distinguish them.

The final query returns respiratory review records with the source encounter identifier, source code and duration. The reasoner supplies the selected review memberships; the asserted graph supplies the original evidence. The test compares exact record identifiers against the accepted source policy. An explanation template records the premises and the two schema steps. No language model is needed for the inference itself.

An AI assistant may help draft a query or explain an answer, but the application still needs parsing, an allowed query surface, execution limits, authorized graph scope, evidence checks and human review of meaning changes. Treat retrieved content as data rather than instructions. Measure whether users can identify an unsupported mapping and reproduce an answer, not only whether the generated prose sounds convincing.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Execute the final evidence query

```
report,derived=reasoning();ds=dataset(derived)
q='''SELECT ?id ?code ?days WHERE {
  GRAPH <urn:graph:derived:v1> { ?r a ex:ReviewCandidate }
  GRAPH <urn:graph:asserted:v1> {
    ?r ex:encounterId ?id; ex:primaryCode ?code; ex:stayDays ?days
  }
} ORDER BY ?id'''
result=list(query(ds,q));display(result[:5])
assert {str(x[0]) for x in result}=={r['encounter_id'] for r in rows() if r['diag_1']=='493'}
```

Attach a reproducible proof sketch

```
explanation={
  'source_fact':'record primaryCode source:493',
  'adapter_policy':'source:493 is categorized as RespiratoryCode',
  'definition':'EncounterRecord and primaryCode some RespiratoryCode -> RespiratoryCodedRecord',
  'workflow_axiom':'RespiratoryCodedRecord subclassOf ReviewCandidate',
  'claim_scope':'Record review routing only',
  'proof_kind':'Auditable proof sketch; not a minimal justification extracted from HermiT'}
display(explanation)
```

Your turn

Return a human-use acceptance task that asks a learner to recover the original source code and distinguish a proposed mapping from an accepted one.

```
answer = None # Replace with your solution
```

Hint: A usability task should test understanding of the evidence boundary.

Explain your model

What would you change if users consistently confuse a review candidate with a confirmed diagnosis?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

E14 · Use a small SWRL rule deliberately

Outcome: Write a positive relational rule, execute it on named source records, and choose when SWRL is appropriate.

Notebook: notebooks/enterprise/E14_use_a_small_swrl_rule_deliberately.ipynb

Time: about 50 minutes.

SWRL expresses an implication: if every body atom holds for a variable binding, the head follows. It is a W3C Member Submission, not an OWL 2 profile or a W3C Recommendation. Unrestricted OWL plus SWRL loses OWL DL's general decidability guarantee. A rule whose head variables appear in the body is range-restricted, but that condition alone is not DL-safety. DL-safe approaches restrict rule variables to an explicitly controlled domain, commonly named individuals, under engine-specific semantics.

This lesson uses the pinned HermiT integration on an isolated rule ontology populated from the same 60 source records. It tests a positive join that relates a respiratory-coded record to the patient reached through its encounter. The result routes information for review; it is not a clinical diagnosis. The rule module remains outside the core OWL 2 DL profile claim. Ordinary OWL class guards do not by themselves prove formal DL-safety, and success on this dataset does not establish support for every SWRL construct.

Prefer ordinary OWL axioms when a subclass, class restriction, inverse or property chain states the intended meaning. Consider SWRL when a supported positive rule needs a variable join that the chosen OWL modeling pattern does not express conveniently and its scope can be controlled. Use SHACL for required fields and data rejection. Use SPARQL for dataset-relative absence, aggregation and explicit transformations. Use Python or an application service for orchestration, external calls, state changes and scheduled actions.

SWRL is not a general workflow engine. Standard rules are monotonic and do not provide ordinary closed-world negation-as-failure, deletion, arbitrary aggregate queries or safe external side effects. Built-ins such as arithmetic and comparison vary by reasoner. Do not assume the `greaterThanOrEqualTo` built-in works merely because a rule string parses. This tested rule uses no built-ins. After removing supporting facts, rebuild the rule ontology from source to avoid stale inferred properties.

Walkthrough setup

Open the notebook and run its bootstrap cell. It locates the course folder and imports the shared helpers; each notebook starts from source data so it can run independently.

Declare the rule vocabulary and load real records

```
from owlready2 import World, Thing, ObjectProperty, Imp, sync_reasoner
world=World()
onto=world.get_ontology('https://example.org/health/rules/')
with onto:
    class EncounterRecord(Thing): pass
    class HospitalEncounter(Thing): pass
    class Patient(Thing): pass
    class Code(Thing): pass
    class RespiratoryCode(Code): pass
    class documents(ObjectProperty): pass
    class hasPatient(ObjectProperty): pass
    class primaryCode(ObjectProperty): pass
    class respiratoryReviewFor(ObjectProperty): pass
    for row in rows():
        record=EncounterRecord('record_'+row['encounter_id'])
        event=HospitalEncounter('encounter_'+row['encounter_id'])
        person=Patient('patient_'+row['patient_nbr'])
        cls=RespiratoryCode if row['diag_1']=='493' else Code
        concept=cls('code_'+row['diag_1'])
        record.documents=[event]
        event.hasPatient=[person]
        record.primaryCode=[concept]
assert sum(len(x.respiratoryReviewFor) for x in EncounterRecord.instances())==0
```

Add the positive join rule and run its supported engine

```
with onto:
    rule=Imp()
    rule.set_as_rule(
        'EncounterRecord(?r), documents(?r, ?e), '
        'HospitalEncounter(?e), hasPatient(?e, ?p), '
        'Patient(?p), primaryCode(?r, ?c), RespiratoryCode(?c) '
        '-> respiratoryReviewFor(?r, ?p)')
print(rule)
sync_reasoner([onto],infer_property_values=True,debug=0)
actual={(r.name.removeprefix('record_'),p.name.removeprefix('patient_'))
        for r in EncounterRecord.instances() for p in r.respiratoryReviewFor}
expected={(r['encounter_id'],r['patient_nbr']) for r in rows() if r['diag_1']=='493'}
assert actual==expected and len(actual)==20
display(sorted(actual)[:5])
```

Your turn

Choose a mechanism for each requirement: required primary-code field, count by code family, supported positive relational join, and a basic class inclusion. Return a dictionary.

```
answer = None # Replace with your solution
```

Hint: The simplest mechanism with the required semantics is usually easiest to govern.

Explain your model

Why does this positive rule not prove anything about records without a respiratory source code? What must be rebuilt when a supporting edge is removed?

Completion evidence

Save the notebook with the exercise passing and write a short explanation in your own words. If a result surprises you, inspect the source values, graph scope and reasoning contract before changing the expected answer.

CHAPTER 27

Capstone review and answer guidance

Use the executed solution notebooks for exact code and output. Compare answer identities and source values, not only a green status or a total count.

Milestone	Expected evidence
Source integrity	Included CSV matches its manifest; reproduction creates identical bytes
Taxonomy	No duplicate preferred labels or hierarchy cycles under project rules
Mapping	Three accepted broad mappings; cross-version proposal remains unasserted
BFO and OWL	Complete pinned BFO import; DL profile passes; ontology consistent
Baseline reasoning	20 review records and 40 cardiopulmonary-coded records
Extension	Exactly 20 cardiac records added to review
Retraction	Removing one respiratory supporting code yields 19 known review records
Query semantics	60 records; 18 A1C categories; 42 missing-category records
Evidence	Every review answer joins to a source encounter ID and code
Product	Catalog metadata, ownership and explicit finite-export contract
Adoption	Measurement plan complete; human pilot fields remain unmeasured

Assess source fidelity, semantic correctness, evidence, reproducibility and user comprehension. Technical checks do not substitute for a measured human pilot.